

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



BÁO CÁO HỌC PHẦN DỰ ÁN

**ĐỀ TÀI: Process Mining trên dữ liệu ngân hàng
BPI-Challenge 2017**

Giảng viên hướng dẫn:

ThS. Nguyễn Thị Thuỳ Linh

Nhóm sinh viên thực hiện

23020329 – Nguyễn Vũ Quang Anh

23020388 – Nguyễn Thế Khôi

Hà Nội, Ngày 5 tháng 6 năm 2026

Lời Cảm Ơn

Chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Nguyễn Thị Thuỳ Linh. Trong suốt quá trình tìm hiểu và thực hiện dự án, cô đã luôn tận tình hướng dẫn, định hướng chuyên môn và cung cấp những góp ý quý báu giúp chúng em vượt qua các khó khăn để hoàn thành tốt học phần này. .

Mặc dù đã nỗ lực hết sức, nhưng do những hạn chế về mặt thời gian và kinh nghiệm thực tiễn, báo cáo chắc chắn khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự châm chước cũng như những nhận xét, góp ý từ cô để nhóm có cơ hội học hỏi và hoàn thiện hơn trong tương lai.

Tóm Tắt Nội Dung

Nghiên cứu này áp dụng các kỹ thuật Process Mining toàn diện trên tập dữ liệu BPI Challenge 2017 nhằm chẩn đoán và tối ưu hóa quy trình xét duyệt khoản vay ngân hàng. Khởi đầu với phân tích nghiệp vụ, kết quả chỉ ra chiến lược phát hành nhiều đề xuất (multi-offer) giúp tăng tỷ lệ chốt giao dịch lên 59% nhưng lại tạo ra nút thắt cổ chai bổ sung hồ sơ, làm chậm tiến độ tổng thể tới 7 ngày. Đánh giá tuân thủ khẳng định quy trình vận hành được thực thi chuẩn chỉ (Trace Fitness đạt 99.30%), đồng thời việc áp dụng Object-Centric Process Mining (OCPM) giúp cung cấp thêm góc nhìn tương tác đa chiều giữa các đối tượng. Cuối cùng, một hệ thống học máy theo hướng đa tiền tố (multi-prefix Early Warning System) được xây dựng để dự đoán kết quả tín dụng từ sớm, đạt hiệu suất ROC-AUC 0.85 tại tiền tố 12. Ngoài ra, dựa trên bài báo về Process Mining của Tóm lại, dự án không chỉ nhận diện chính xác nguyên nhân cốt lõi gây chậm trễ hiệu suất, mà còn cung cấp giải pháp dự báo chủ động nâng cao năng lực quản trị rủi ro.

Mục lục

1	Giới thiệu chung về Process Mining	3
1.1	Process Mining là gì?	3
1.2	Tại sao nên Khai phá dữ liệu quy trình	3
2	Giới thiệu bài toán và Dữ liệu	3
2.1	Giới thiệu về Dataset BPI2017-challenge	3
2.2	Tổng quan về Dataset và Phân tích Khám phá (EDA)	5
2.3	Đặt Vấn Đề	7
2.4	Mục tiêu dự án	7
2.5	Cách tiếp cận:	7
3	Phân tích và khai phá quy trình	8
3.1	Nhận diện hiện tượng Concept Drifts	8
3.1.1	Mục tiêu	8
3.1.2	Thuật toán áp dụng	8
3.2	Phân tích Nghiệp vụ và Nút thắt (Bottleneck Analysis)	11
3.2.1	Mục tiêu	11
3.2.2	Phương pháp sử dụng	11
3.3	Kiểm tra Tuân thủ Quy trình (Conformance Checking) và tìm lí do chủ yếu gây Bottleneck	11
3.3.1	Phân Tích Thời Gian Chờ (Bank Wait Time vs Customer Wait Time)	13
3.4	Tích hợp Object-Centric Process Mining (OCPM) và Phân tích Đồ thị DFG	14
3.4.1	Giải quyết bài toán Spaghetti Process bằng thuật toán phân nhóm OCPM	14
3.4.2	Xây dựng Directly-Follows Graph (DFG) thông qua PM4PY	16
4	Các bài toán mở rộng	17
4.1	Bài toán cảnh báo sớm	17
4.1.1	Mục tiêu	17
4.1.2	Phương pháp	17
4.2	Thuật toán xây dựng Happy Path	18
5	Kết quả	19
5.1	Bài toán cảnh báo sớm	19
6	Kết luận và Hướng phát triển	19

1 Giới thiệu chung về Process Mining

1.1 Process Mining là gì?

Process Mining hay là khai phá dữ liệu quy trình, là một lĩnh vực giao thoa giữa Khoa học dữ liệu (Data Science) và Quản lý Quy trình Nghiệp vụ (Business Process Management - BPM). Khái niệm này ban đầu nghe có vẻ trừu tượng, nhưng về bản chất, nó là việc sử dụng dữ liệu thực tế để vẽ ra bức tranh chính xác về cách mọi thứ đang thực sự vận hành, thay vì dựa vào cảm tính hay những quy trình được vẽ trên giấy.

Input của bài toán là những Event logs ghi lại nhật ký khi có hành động xảy ra. Một event logs phải có những yếu tố cơ bản như:

- Case ID (Mã ca/Hồ sơ): Định danh duy nhất cho một quy trình cụ thể
- Activity (Hoạt động): Bước công việc vừa được thực hiện
- Timestamp (Dấu thời gian): Thời điểm chính xác sự kiện đó xảy ra

Việc thu thập và trích xuất hàng triệu dòng log này từ các nguồn dữ liệu phức tạp hoặc kho dữ liệu lưu trữ tốc độ cao chính là bước đầu tiên và quan trọng nhất.

1.2 Tại sao nên Khai phá dữ liệu quy trình

Trong cách quản lý truyền thống, khi muốn tối ưu quy trình, người ta thường phải phỏng vấn từng nhân viên để vẽ ra quy trình (rất tốn thời gian, chủ quan và thường không chính xác).

Process Mining thay đổi hoàn toàn điều đó bằng cách mang tư duy Data-driven (Dẫn dắt bằng dữ liệu) vào quản lý quy trình. Nó cung cấp sự minh bạch tuyệt đối. Nếu một hệ thống có lỗi hoặc một chuỗi sự kiện gây ra độ trễ lớn, dữ liệu logs sẽ thể hiện ra ngay lập tức khi chúng ta EDA.

Hơn nữa, khi kết hợp với Machine Learning, Process Mining không chỉ dừng lại ở việc phân tích quá khứ mà còn có thể dự đoán (Predictive Process Monitoring), ví dụ như dự đoán xem một đơn hàng cụ thể có nguy cơ bị trễ hạn hay không ngay khi nó vừa bước vào hệ thống.

2 Giới thiệu bài toán và Dữ liệu

2.1 Giới thiệu về Dataset BPI2017-challenge

Tập dữ liệu BPI Challenge 2017 (BPIC 2017) là một trong những bộ sự kiện (event logs) chuẩn mực và phổ biến nhất trong lĩnh vực Process Mining. Dữ liệu này được trích xuất từ hệ thống quản lý quy trình cấp phát khoản vay cá nhân của một tổ chức tài chính tại Hà Lan. Nó ghi lại toàn bộ vòng đời của một hồ sơ vay vốn, từ lúc khách hàng nộp đơn yêu cầu cho đến khi khoản vay được giải ngân, bị từ chối hoặc bị khách hàng hủy bỏ.

Điểm đặc biệt của BPIC 2017 là tính phức tạp trong nghiệp vụ, thể hiện qua sự tương tác giữa ba luồng sự kiện (event origins) đan xen nhau trong cùng một hồ sơ:

- **A-events (Application):** Nhóm sự kiện thể hiện trạng thái tổng thể của hồ sơ vay (ví dụ: *A_Create_Application*, *A_Submitted*, *A_Denied*).
- **O-events (Offer):** Nhóm sự kiện liên quan đến các đề xuất/gói vay được ngân hàng gửi cho khách hàng (ví dụ: *O_Created*, *O_Sent*, *O_Accepted*). Một hồ sơ (Case) có thể có nhiều đề xuất khác nhau nếu khách hàng muốn thương lượng lại.
- **W-events (Workflow):** Nhóm sự kiện ghi lại các công việc thực tế mà nhân viên tín dụng hoặc hệ thống phải xử lý (ví dụ: *W_Call_after_offers*, *W_Validate_application*).

Về mặt cấu trúc, bộ dữ liệu lưu trữ các trường thông tin đặc biệt quan trọng phục vụ cho việc chẩn đoán quy trình và xây dựng mô hình dự báo, được chia thành hai nhóm chính:

1. Các thuộc tính cấp độ hồ sơ (Case Attributes): Đây là các thông tin tĩnh, gắn liền với toàn bộ vòng đời của một hồ sơ vay (Case).

- **Case ID (*case:concept:name*):** Mã định danh duy nhất cho mỗi hồ sơ vay. Đây là "chìa khóa" bắt buộc để xâu chuỗi các sự kiện rời rạc thành một chuỗi quy trình hoàn chỉnh (Trace).
- **RequestedAmount (*case:RequestedAmount*):** Số tiền mà khách hàng yêu cầu vay ban đầu. Đây là biến số quan trọng để phân tích sự khác biệt trong chiến lược xét duyệt hoặc rủi ro giữa các khoản vay lớn và nhỏ.
- **LoanGoal (*case:LoanGoal*):** Mục đích của khoản vay (ví dụ: Mua ô tô, sửa nhà, chi tiêu cá nhân), giúp phân loại và đánh giá thời gian xử lý theo từng nhóm sản phẩm.

2. Các thuộc tính cấp độ sự kiện (Event Attributes): Đây là thông tin động, được ghi lại chi tiết tại mỗi bước thực thi trong hệ thống.

- **Activity (*concept:name*):** Tên của bước công việc hoặc trạng thái vừa diễn ra (VD: *W_Validate_application*). Trường này là đầu vào cơ bản nhất cho thuật toán Process Discovery.
- **Timestamp (*time:timestamp*):** Thời gian chính xác sự kiện bắt đầu hoặc kết thúc. Đây là yếu tố cốt lõi để tính toán thời gian luân chuyển (lead time) và tìm kiếm nút thắt cổ chai (bottleneck analysis).
- **Resource (*org:resource*):** Định danh của nhân sự, phòng ban hoặc hệ thống tự động thực thi sự kiện. Trường này đặc biệt có giá trị để kiểm tra hiệu suất làm việc của nhân viên và đánh giá sự phân bổ nguồn lực.

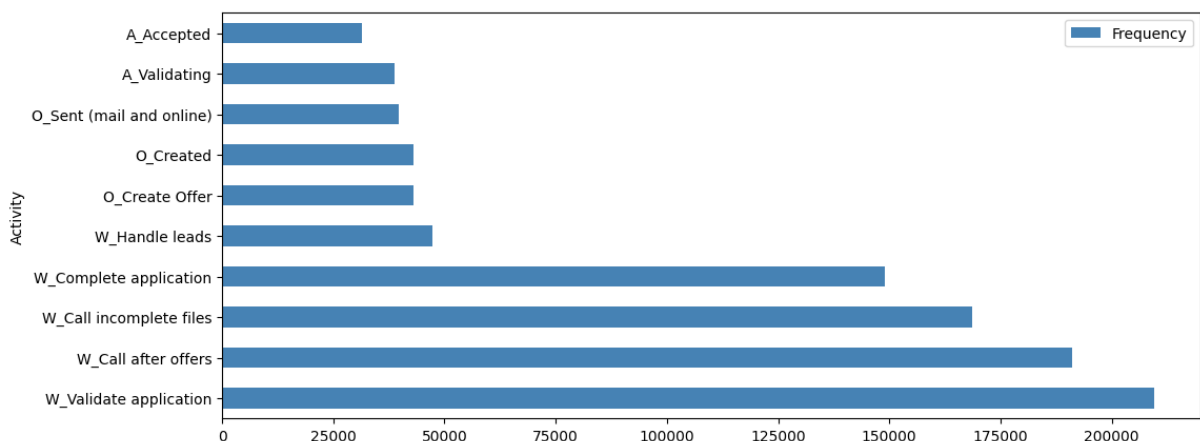
- **Lifecycle Transition (*lifecycle:transition*):** Trạng thái vòng đời của công việc (thường là *start*, *complete*, *suspend*, *resume*). Việc phân tách rõ thời điểm bắt đầu và kết thúc cho phép đo lường chính xác thời gian thực thi (processing time) so với thời gian chờ (waiting time).

2.2 Tổng quan về Dataset và Phân tích Khám phá (EDA)

Quá trình phân tích khám phá dữ liệu (EDA - Exploratory Data Analysis) trên tập dữ liệu BPI Challenge 2017 đóng vai trò nền tảng để hiểu rõ luồng vận hành thực tế. Với quy mô 1.201.090 sự kiện (events) trải trên 31.509 hồ sơ (cases), quá trình EDA đã bóc tách 26 loại Activity và phân loại chúng thành 3 nhóm sự kiện cốt lõi:

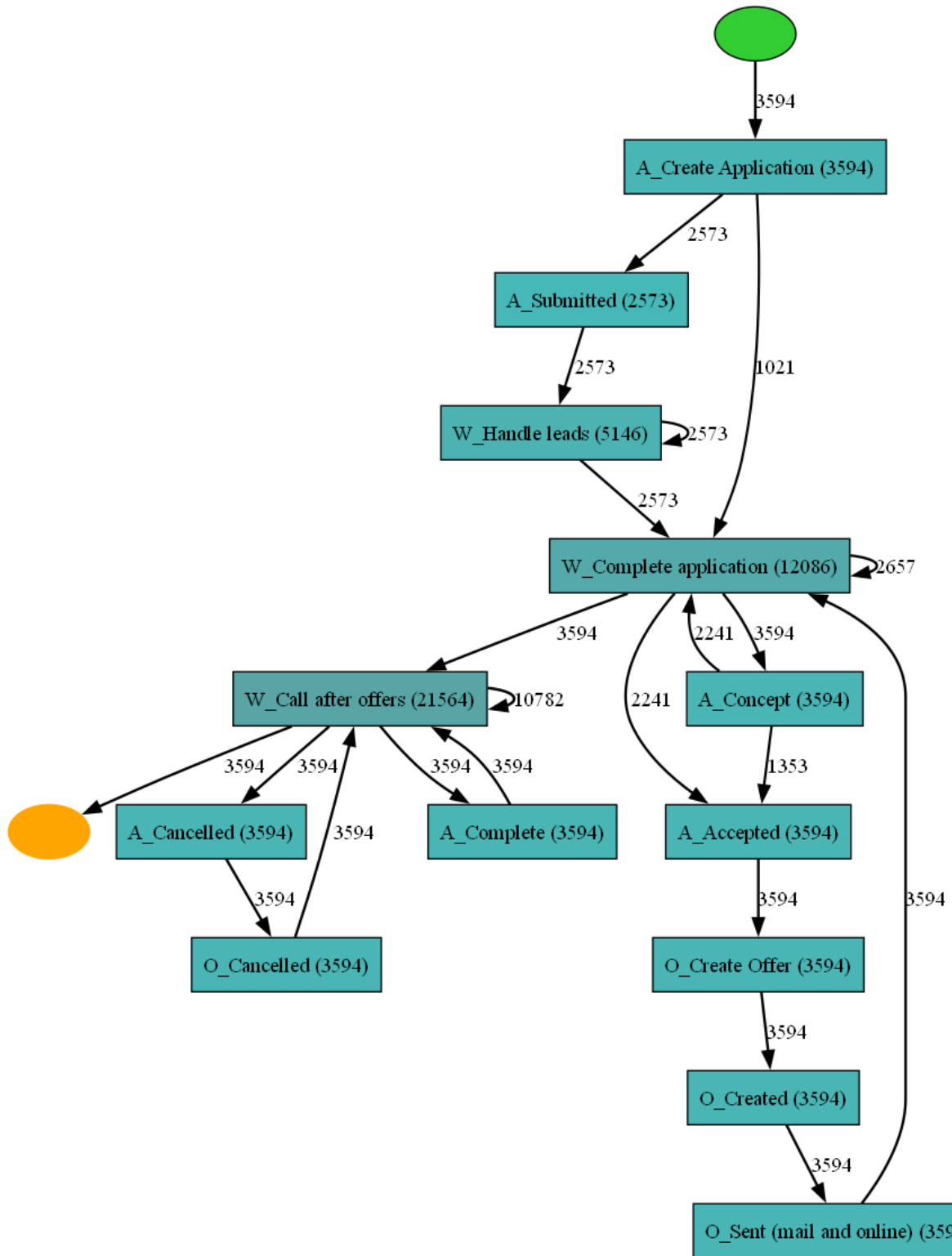
- **Nhóm A_ (Application State):** Thể hiện các mốc trạng thái chính của một hồ sơ vay, từ lúc khởi tạo (*A_Create Application*), quá trình thẩm định (*A_Validating*), cho đến các quyết định cuối cùng như chấp nhận (*A_Accepted*), từ chối (*A_Denied*) hoặc khách hàng tự hủy (*A_Cancelled*).
- **Nhóm O_ (Offer State):** Ghi nhận các tương tác liên quan đến gói đề xuất khoản vay (*O_Create Offer*, *O_Sent*, *O_Returned*). Điểm đặc biệt bộc lộ từ dữ liệu là một hồ sơ có thể xuất hiện nhiều vòng lặp sinh đề xuất mới.
- **Nhóm W_ (Workflow Action):** Phản ánh các tác vụ thủ công của nhân viên tín dụng thao tác trên hệ thống quản lý, ví dụ như gọi điện chăm sóc khách hàng (*W_Call after offers*) hoặc kiểm tra giấy tờ (*W_Validate application*).

Các hoạt động nhiều nhất được thống kê từ bộ dữ liệu được biểu diễn thông qua subplot như hình dưới đây



Hình 1: Biểu đồ thể hiện tần suất các hoạt động trong dataset

Ngoài ra với công cụ PM4PY và thuật toán inductive miner, ta có thể xây dựng được một FPtree với nội dung top 5 biến thể luồng quy trình phổ biến nhất để ta có thể nhìn thấy được cái nhìn tổng quan của cách mà ngân hàng vận hành ở trong dataset này



Hình 2: top 5 biến thể (luồng đi) phổ biến nhất

Qua các thống kê mô tả và biểu diễn phân bố (Distribution Analysis) được phân tích trong Dự Án, EDA chỉ ra rằng phần lớn các hồ sơ đi theo những biến thể (variants) tiêu chuẩn, nhưng có sự phân tán rất lớn về thời gian hoàn thành (Case Duration). Đặc biệt, tần suất lặp đi lặp lại của các sự kiện như **A_Incomplete** (thiếu hồ sơ) và **A_Validating** đã sớm báo hiệu sự tồn tại của các vòng lặp (ping-pong behavior) trong khâu bổ sung giấy

từ. Đây là tiền đề quan trọng định hướng cho quá trình chẩn đoán nút thắt (Bottleneck Analysis) chuyên sâu. Các phân tích chuyên sâu sẽ được trình bày chi tiết trong mục Phân tích và khai phá quy trình

2.3 Đặt Vấn Đề

Các bài toán được tìm hiểu và thực nghiệm dựa trên dataset BPI2017 để nhằm hiểu rõ nút thắt của quy trình của ngân hàng:

1. **Bài toán Hiệu suất và Nút thắt (Performance Bottleneck):** Đặt vấn đề về tính hai mặt của chiến lược Multi-offer (tăng doanh thu nhưng gây kẹt hệ thống) và nhu cầu bóc tách nguyên nhân gây chậm trễ (từ phía khách hàng hay ngân hàng).
2. **Bài toán Tuân thủ Quy trình (Conformance Checking):** Nêu câu hỏi liệu sự chậm trễ này là do nhân viên làm ẩu/vi phạm quy trình hay do bản thân thiết kế quy trình chưa tốt
3. **Bài toán Dự báo Rủi ro Sớm (Predictive Analytics):** Đặt vấn đề về sự lãng phí tài nguyên khi đợi đến cuối cùng mới từ chối hồ sơ, từ đó dẫn dắt đến nhu cầu xây dựng mô hình Học máy Cảnh báo sớm (Early Warning System).
4. **Bài toán cause-effects:** Dựa vào mối quan hệ nhân quả để tìm ra concept drift và hướng phát triển tương lai là xây dựng mô hình dự báo 'Nguyên Nhân' dựa trên dữ liệu giả lập từ "HAPPY PATH"

2.4 Mục tiêu dự án

Thấu hiểu được cách vận hành của ngân hàng trong dataset. Ngoài ra theo dõi quan sát quy trình xung quanh điểm drift. Giải thích lí do vì sao dẫn tới bottleneck trong thực tế và hướng tối ưu quy trình. Xây dựng thuật toán cảnh báo rủi ro sớm. Khám phá mối quan hệ, trích xuất thông tin dữ liệu bằng OCPM

2.5 Cách tiếp cận:

Sử dụng các công nghệ, thuật toán khai phá quy trình thông qua thư viện PM4PY, trích xuất dữ liệu đối tượng thành OCEL, phân tích dữ liệu và trực quan hoá bằng pandas, matplotlib và seaborn. Xây dựng hệ thống học máy bằng các thư viện tương ứng và đánh giá dựa trên dữ liệu.

3 Phân tích và khai phá quy trình

3.1 Nhận diện hiện tượng Concept Drifts

3.1.1 Mục tiêu

Rà soát pháp y dữ liệu nhằm giải mã các biến động ngầm (Concept Drifts) trong quy trình xét duyệt hồ sơ tín dụng năm 2016. Thay vì đánh giá tổng thể hệ thống, phân tích được cô lập nghiêm ngặt vào hoạt động do con người trực tiếp thực hiện `W_Validate application`, nhằm loại bỏ nhiễu từ các tác vụ tự động và bóc tách chính xác "Thời gian thực làm"(True Service Time) khỏi "Thời gian chờ"(Turnaround Time).

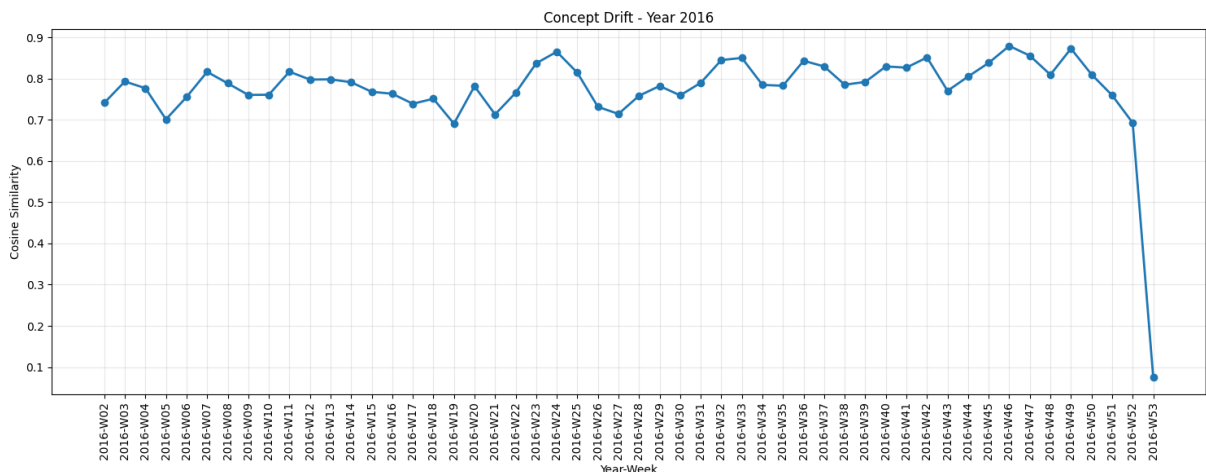
3.1.2 Thuật toán áp dụng

Giai đoạn 1: Tạo ra tín hiệu bằng Cosine Similarity

Event logs nguyên bản chỉ là những dòng chữ (Activity A, Activity B...) và thời gian (Timestamp). Máy tính không thể trực tiếp tính toán đứt gãy trên chữ viết. Nó cần một thước đo khoảng cách.

- : Ta cắt dữ liệu thành từng khung thời gian (Ví dụ: Tuần 1, Tuần 2...).
- : Đếm tần suất xuất hiện của các chuỗi sự kiện trong mỗi tuần, biến mỗi tuần thành một Vector toán học (Feature Vector).
- : Ta dùng Cosine Similarity để đo góc lệch giữa Vector Tuần 1 và Vector Tuần 2.

Kết quả:



Hình 3: Biểu đồ góc lệch giữa tuần hiện tại và tuần trước đó

Nhận xét:

- : Từ tuần 1 tới 18 ta thấy đều duy trì ổn định ở mức 0.8, riêng tuần 5 có sự sụt lún là do hồ sơ tồn đọng từ năm trước và sự kiện lễ hội lớn của Hà Lan.

- : Ở tuần 19 đường Cosine rơi xuống 1 nhịp, bắt đầu sự bất ổn so với các tuần trước đó.
- : Ở tuần 21 ta đã thấy sự thay đổi so với quy trình của tuần 19 và giữ vững tới tuần 24.
- : TỪ tuần 24 trở đi ta thấy sự trượt dốc không phanh, quy trình có sự thay đổi mạnh tới tuần 27. Các tuần sau đó đã ổn định đáng kể.
- : Tại tuần 49, quy trình đã ổn định trước đó bắt đầu thay đổi đáng kể liên tiếp ở các tuần sau. Đáng kể là ở tuần cuối cùng, ta có thể nhận thấy là do bắt đầu chính sách mới được vận hành bắt đầu từ 2017 nên đã không còn giống 2016.

Giai đoạn 2: Định vị điểm gãy bằng PELT

Để vượt qua giới hạn của các phương pháp quan sát bằng mắt thường, quy trình sử dụng chuỗi thuật toán chuyên sâu: Phát hiện điểm gãy (Change Point Detection): Thuật toán PELT (Pruned Exact Linear Time) với mô hình phạt L2 được triển khai trên dữ liệu đã chuẩn hóa, nhằm định vị chính xác mốc thời gian phân phối dữ liệu bị xô lệch vĩnh viễn.

Để hiểu rõ PELT, hãy tưởng tượng luồng dữ liệu sự kiện (như chuỗi Khối lượng công việc hay Thời gian xử lý) là một tín hiệu liên tục chạy theo thời gian. Nhiệm vụ của chúng ta là rà soát toàn bộ dải tín hiệu đó để tìm ra các mốc mà tại đó, cấu trúc hệ thống bị xô lệch vĩnh viễn (Concept Drift).

Thuật toán PELT (được Killick, Fearnhead và Eckley công bố năm 2012) giải quyết bài toán này dựa trên 3 trụ cột cốt lõi làm nên tên tuổi của nó:

• Exact

Nhiều kỹ thuật phát hiện điểm gãy trong Process Mining (như Control Charts hay Sliding Windows) dựa trên phương pháp xấp xỉ (heuristics) hoặc tự đặt ngưỡng cảnh báo. PELT không làm vậy. Nó tiếp cận bài toán dưới dạng tối ưu hóa toàn cục (Global Optimization). Mục tiêu của PELT là tìm ra số lượng điểm gãy m và vị trí chính xác của chúng $\tau = (\tau_1, \tau_2, \dots, \tau_m)$ sao cho tổng chi phí (Cost) chia cắt chuỗi tín hiệu là nhỏ nhất. Công thức tối ưu hóa được định nghĩa nghiêm ngặt như sau:

$$\min_{m, \tau} \left\{ \sum_{i=1}^{m+1} \mathcal{C}(y_{(\tau_{i-1}+1):\tau_i}) + \beta m \right\}$$

Trong đó:

- $\mathcal{C}$ là hàm chi phí (Cost function). Trong các phân tích vừa qua, chúng ta dùng model="l2" (hàm phạt L2). Nó đo lường tổng bình phương sai số, một thước đo cực kỳ nhạy bén để bắt được sự chuyển pha của giá trị trung bình trong hệ thống.

- β là hệ số phạt (Penalty). Nếu không có β , thuật toán sẽ cấm điểm gây ở khắp mọi nơi khiến tổng $\mathcal{C}$ tiến về 0 (hiện tượng Overfitting). β đóng vai trò như một lực cản, buộc thuật toán chỉ được phép "gãy" khi sự biến đổi cấu trúc đủ mạnh để bù đắp lại hình phạt này.

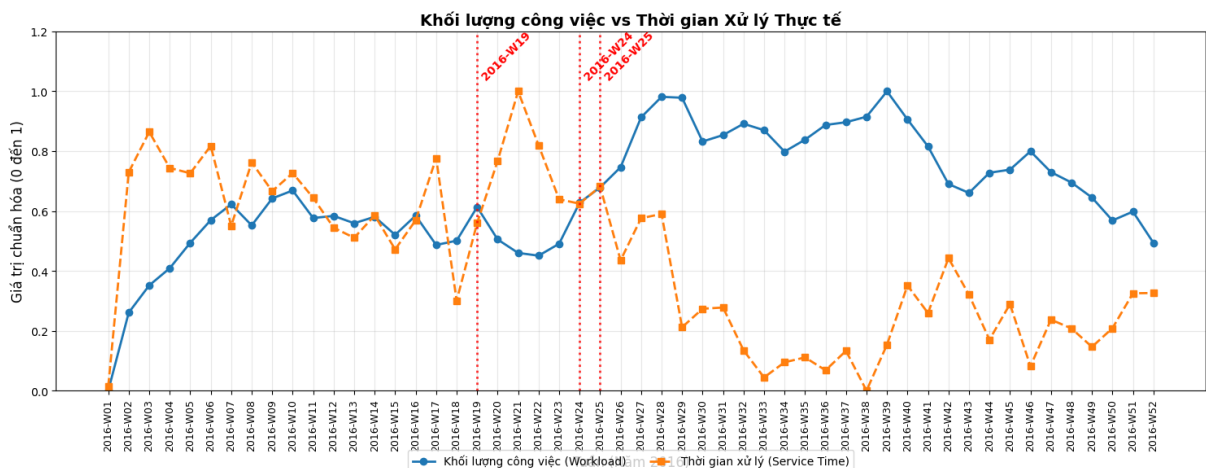
• Pruned

Để giải được phương trình tối ưu tuyệt đối ở trên bằng quy hoạch động (Dynamic Programming), hệ thống thường phải so sánh mọi điểm trong quá khứ với điểm hiện tại. Nếu bạn có một bản log hàng triệu sự kiện, thuật toán truyền thống có độ phức tạp thời gian là $\mathcal{O}(N^2)$ sẽ khiến máy tính treo vĩnh viễn. Đây là lúc sự thiên tài của đặc tính Pruned (Tỉa cành) xuất hiện. Thuật toán dựa trên một định lý loại trừ xuất sắc: Nếu tại một thời điểm t , một điểm cắt dự tuyến s (trong quá khứ) đã chứng minh được rằng nó tạo ra một cấu trúc chi phí tồi tệ hơn so với điểm cắt tại chính t , thì s sẽ KHÔNG BAO GIỜ có cơ hội trở thành điểm cắt tối ưu cho bất kỳ thời điểm nào trong tương lai. Nhờ quy luật này, thay vì ôm đồm toàn bộ lịch sử, PELT liên tục sàng lọc và "tỉa" bỏ những điểm quá khứ không còn tiềm năng khi nó quét dọc theo trục thời gian.

• Linear Time

Nhờ việc loại bỏ liên tục các tập ứng viên vô dụng bằng kỹ thuật tỉa cành, khối lượng tính toán được nén lại đáng kinh ngạc. PELT kéo độ phức tạp của bài toán từ $\mathcal{O}(N^2)$ xuống chỉ còn $\mathcal{O}(N)$. Nó xử lý dữ liệu với tốc độ tuyến tính, quét nhanh gọn các tập Event Logs khổng lồ mà vẫn đảm bảo nghiệm trả về là tối ưu và chính xác tuyệt đối.

Kết quả:



Hình 4: Khối lượng công việc và thời gian xử lý

Nhận xét:

- Các tuần đầu thời gian xử lý rất lớn nên số lượng hồ sơ bị tồn đọng cộng dồn lên. Sau đó được ổn định tới tuần 18.

- Tại tuần 19 ta thấy một đỉnh cục bộ ở khối lượng công việc và sau đó là sự bùng nổ về thời gian xử lý mặc dù khối lượng công việc các tuần sau đã giảm.
- Từ tuần 24 trở đi ta thấy khối lượng công việc bắt đầu tăng đột biến nhưng ngược lại là thời gian xử lý lại giảm mạnh. Điều này cho thấy quy trình đã được thay đổi để phù hợp hơn.
- Mặt khác ta có thể thấy là sự đối lập trong hình 4 là khi khối lượng công việc giảm thì thời gian xử lý lại tăng và ngược lại. Điều này có thể chứng minh khi lượng hồ sơ giảm thì doanh nghiệp đã tăng mức thận trọng trong mỗi hồ sơ dẫn đến xử lý lâu hơn, còn khi lượng công việc tăng cao, họ đã bỏ bớt, cắt xén quy trình để tạo sự thông thoáng hơn, dẫn tới việc hồ sơ được giải quyết nhanh hơn.
- Ta có thể thấy rằng việc gia tăng khối lượng công việc ở tuần 24 đã dẫn tới sự giảm sút thời gian xử lý ở tuần 29, nhóm đã tính được $p\text{ value} = 0.081658$, tuy rằng gấp 11 lần SOTA nhưng cũng có thể phần nào chứng minh được giả thuyết đó.

3.2 Phân tích Nghiệp vụ và Nút thắt (Bottleneck Analysis)

3.2.1 Mục tiêu

Mục tiêu của phần này là nhận diện các điểm nghẽn (bottleneck) làm giảm hiệu suất luân chuyển hồ sơ trong hệ thống, thông qua việc tính toán Thời gian luân chuyển (Turnaround Time/Lead Time) và phân tách thời gian chờ (Waiting Time) giữa các sự kiện.

Qua quá trình phân tích nhật ký sự kiện, kết quả cho thấy chiến lược phát hành "nhiều đề xuất vay cùng lúc" (Multi-offer) mang lại lợi ích về mặt doanh thu nhưng lại là nguyên nhân chính gây ra nút thắt cổ chai về mặt vận hành. Cụ thể:

Điểm nghẽn tại khâu bổ sung hồ sơ: Việc cho phép khách hàng thương lượng và ngân hàng gửi nhiều offer liên tiếp khiến luồng sự kiện O-events bị lặp lại nhiều lần. Quá trình giao tiếp và chờ khách hàng bổ sung giấy tờ cho các offer này đã tạo ra một khoảng thời gian chờ khổng lồ.

Hậu quả lên tiến độ tổng thể: Phân tích định lượng chỉ ra rằng, nút thắt tại khâu này đã làm chậm tiến độ luân chuyển của toàn bộ quy trình lên tới trung bình 7 ngày cho mỗi hồ sơ.

Đề xuất: Mặc dù tỷ lệ chốt giao dịch (conversion rate) tăng lên 59

3.2.2 Phương pháp sử dụng

3.3 Kiểm tra Tuân thủ Quy trình (Conformance Checking) và tìm lí do chủ yếu gây Bottleneck

Dựa vào bộ dữ liệu và các phương pháp thống kê trong dự án, ta có thể thấy được là trong ngân hàng này thì multi-offer được ưa chuộng hơn single-offer rất nhiều. Vậy multi-offer và single-offer là những gì:

- Multi-offer: Multi-offer (Nhiều đề xuất): Là những hồ sơ mà ngân hàng linh hoạt đàm phán với khách hàng và phát hành từ 2 offer trở lên (có thể sửa đổi hạn mức, lãi suất cho phù hợp với mong muốn của khách).
- Single-offer: Là những hồ sơ (case) mà ngân hàng chỉ phát hành duy nhất 1 offer. Khách hàng hoặc là đồng ý ký nhận, hoặc là từ chối/hủy bỏ. Quá trình đàm phán kết thúc nhanh chóng.

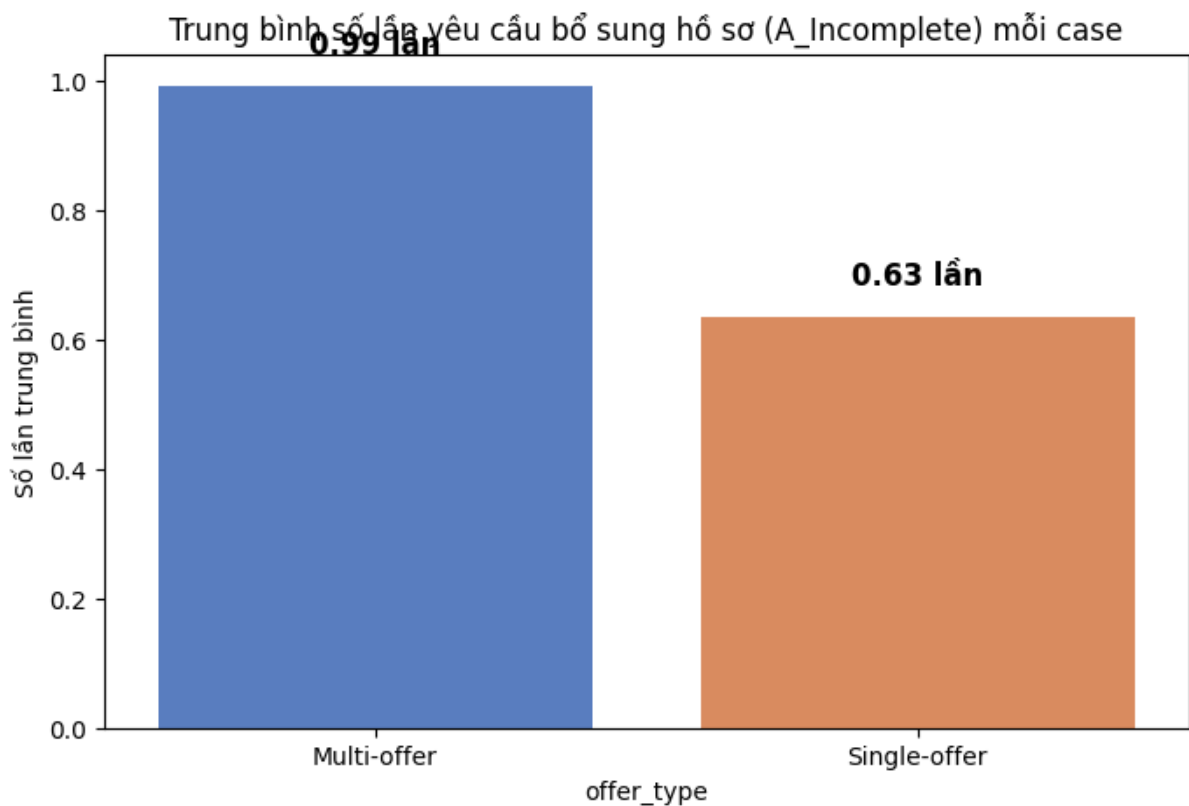
Khi chạy phân tích trên 31.509 hồ sơ, dữ liệu trả về như sau:

- Số lượng hồ sơ Single-offer chiếm áp đảo với 22.950 hồ sơ.
- Số lượng hồ sơ Multi-offer ít hơn, đạt 8.559 hồ sơ

Tỷ lệ chấp nhận (Acceptance Rate) - tức là tỷ lệ khách hàng đồng ý chốt khoản vay và ngân hàng giải ngân thành công. Kết quả chỉ ra một sự chênh lệch rõ ràng:

- Single-offer: Tỷ lệ chốt sale chỉ đạt 53.1%.
- Multi-offer: Tỷ lệ chốt sale tăng vọt lên 59.0

Nhận xét: Việc ngân hàng linh hoạt đàm phán và đưa ra nhiều đề xuất thay thế (Multi-offer) giúp giữ chân khách hàng rất tốt, tuy nhiên với việc đa phần khách hàng thích multi-offer thì nó cũng là lí do gây nghẽn dữ liệu vì đa phần họ sẽ bị thiếu hồ sơ



Hình 5: Trung bình yêu cầu bổ sung của 2 loại offer

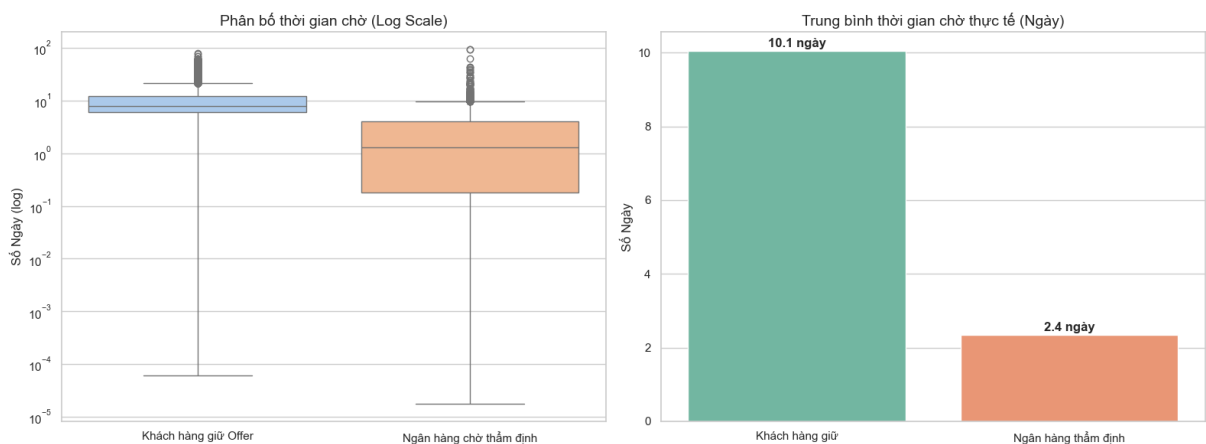
Nút thắt cổ chai (Bottleneck) khổng lồ cho hệ thống vận hành (Back-office):

- Khi phát hành nhiều offer, quy trình thường xuyên rơi vào trạng thái lỗi thiếu hồ sơ, thiếu giấy tờ chứng minh (A_Incomplete).
- Chính sự chấp vá, bổ sung hồ sơ lặp đi lặp lại này khiến tổng thời gian chờ đợi (Waiting time) của một hồ sơ Multi-offer bị chậm hơn tới 7 ngày so với một quy trình chuẩn (từ 13 ngày kéo dài lên 20 ngày)
- Dựa trên thống kê từ dataset, ta còn thấy được 62.8% dữ liệu là khách hàng phải đi gom hồ sơ; còn lại 37.2% là ngân hàng phải review hồ sơ. Thông qua quá trình Comformance Checking, đã chứng minh được gần 100% ngân hàng thực hiện đúng quy trình vì họ rất khi thay đổi nên đa phần các khoảng thời gian tạo bottleneck do có quá nhiều hồ sơ tắc nghẽn

3.3.1 Phân Tích Thời Gian Chờ (Bank Wait Time vs Customer Wait Time)

Customer Wait Time (Khách hàng ngâm): Thời gian từ lúc ngân hàng gửi Offer (O_Sent (mail and online)) đầu tiên đến lúc nhận được phản hồi từ khách (O_Returned) đầu tiên.

Bank Wait Time (Ngân hàng duyệt): Thời gian từ lúc nhận phản hồi ('O_Returned') đến lúc ngân hàng đưa ra phán quyết sau khi thẩm định (Có thể là 'A_Incomplete' - thiếu hồ sơ, 'A_Pending' - chờ giải ngân, 'A_Accepted', hoặc 'A_Denied').



Hình 6: Biểu đồ hộp và biểu đồ cột thể hiện thời gian chờ

Nhận xét:

1. Khách hàng là nguyên nhân chính gây ra độ trễ (Bottleneck) Biểu đồ cột (bên phải) cho thấy rõ sự chênh lệch: Khách hàng ngâm/giữ Offer trung bình tốn tới 10.1 ngày, chậm gấp hơn 4 lần so với thời gian ngân hàng dành ra để thẩm định (2.4 ngày).
2. Khách hàng giữ Offer: Phần hộp màu xanh nằm ở dải giá trị rất cao (xoay quanh mốc 10¹ tức 10 ngày), xuất hiện nhiều điểm dị biệt (outliers) kéo dài lên sát mốc 100 ngày. Điều này chứng tỏ hành vi phản hồi của khách hàng rất khó lường và thường xuyên bị kéo dài. Ngân hàng chờ thẩm định: Phần hộp màu cam nằm thấp

hơn hẳn (xoay quanh mốc 10^0 tức 1 ngày), cho thấy thời gian xử lý nội bộ của ngân hàng phần lớn là nhanh chóng, ổn định và tập trung.

=> Ngân hàng đang làm khá tốt nhiệm vụ của mình (xử lý nhanh). Lỗi hổng lớn nhất làm chậm toàn bộ quy trình giải ngân nằm ở khâu chờ khách hàng bổ sung, ký tá và hoàn nộp lại hồ sơ. Ngân hàng cần có các giải pháp tự động nhắc nhở (follow-up) khách hàng gặt gao hơn ở giai đoạn này để rút ngắn thời gian.

3.4 Tích hợp Object-Centric Process Mining (OCPM) và Phân tích Đồ thị DFG

Sự khác biệt cốt lõi của dự án nằm ở việc kết hợp tư duy phân rã đối tượng (OCPM) với các thuật toán vẽ đồ thị (DFG) và cửa sổ trượt thời gian (Local-Window Drift) để tìm ra nguyên nhân gốc rễ của sự chậm trễ. Cách tiếp cận thuật toán được chia thành 2 giai đoạn như sau:

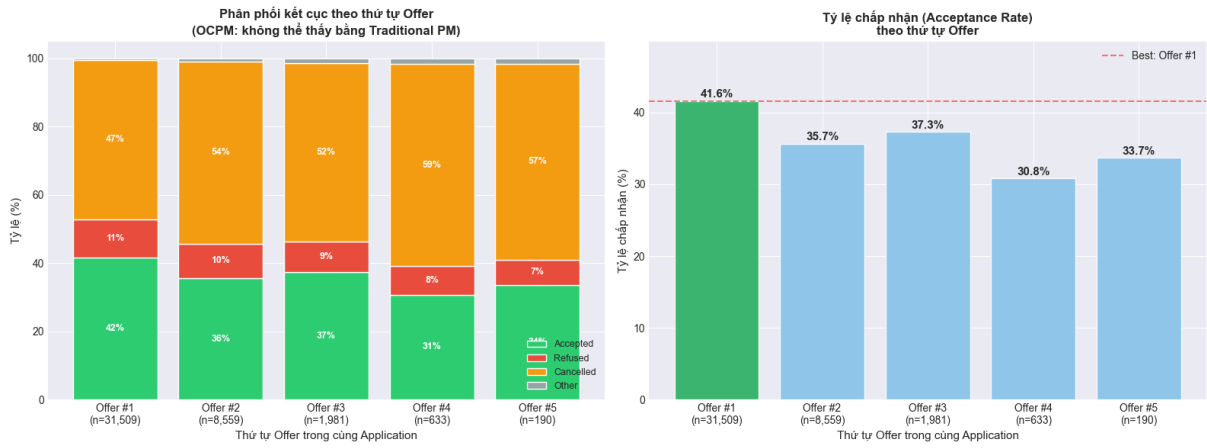
3.4.1 Giải quyết bài toán Spaghetti Process bằng thuật toán phân nhóm OCPM

Trong Process Mining truyền thống, khi ép buộc toàn bộ sự kiện vào một mã hồ sơ (Application), mối quan hệ 1-Nhiều (1 Application sinh ra nhiều Offer) tạo ra sự bùng nổ tổ hợp đường đi, dẫn tới biểu đồ rối rắm như mạng nhện (spaghetti process). Bằng tư duy **OCPM**, hệ thống thực hiện hai bước thuật toán để bóc tách:

- **Phân mảnh đối tượng (Object Stratification):** Tách biệt hoàn toàn vòng đời cốt lõi của Application ra khỏi các luồng Offer độc lập. Nhóm các hồ sơ theo từng loại nghiệp vụ (ApplicationType).
- **Lọc biến thể (Variant Coverage Algorithm):** Thuật toán quét và chỉ giữ lại 65% các biến thể (variants) xuất hiện phổ biến nhất (Top-K Coverage). Điều này giúp tự động loại bỏ các hồ sơ bị biến dạng do những vòng lặp Offer bất thường, làm sạch tập dữ liệu đầu vào.

Bên cạnh đó OCPM còn giúp chúng ta khắc phục được những gì traditional PM chưa làm được như 2 insight mà ta có thể khai thác như sau:

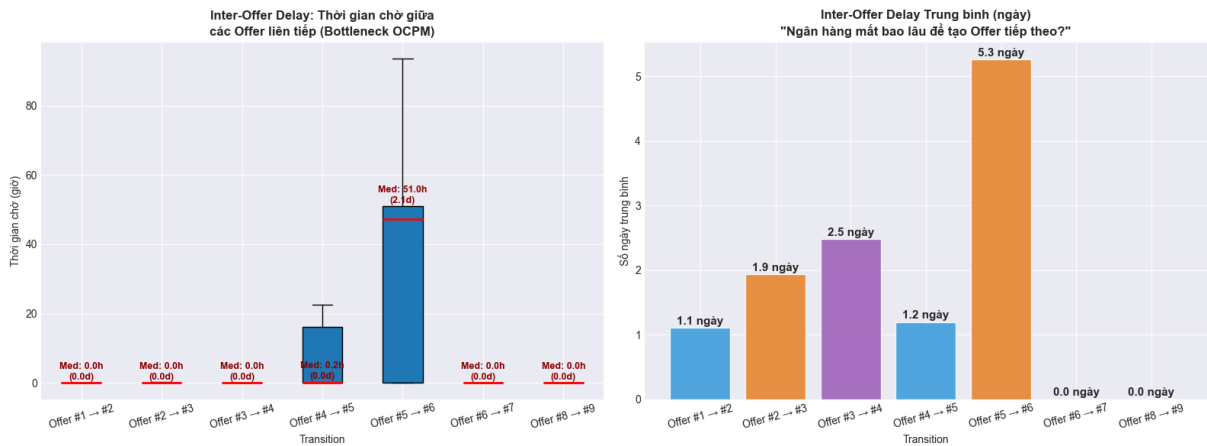
1. Offer thứ mấy thì sẽ được chấp nhận nhiều nhất?



Hình 7: Thống kê offer

Vì traditional PM sẽ coi tất cả các offer là 1 trace thay vì tách rõ ra như OCPM, từ đây ta có thể thấy Offer #1 sẽ mang lại tỉ lệ thành công cao nhất => Ngân hàng có thể cải thiện ở khâu này khai thác triệt để thêm nâng tỉ lệ khách hàng chọn 1 offer cũng là 1 hướng phát triển vừa nâng tỉ lệ thành công của 1 case mà còn giảm thiểu được thời gian xử lý vì đa phần nghìn diễn ra ở multi offer.

2. Thời gian chờ giữa các offer là bao nhiêu ?



Hình 8: Khoảng thời gian chờ giữa từng offer

Hypothesis: Có một khoảng chờ (bottleneck) đáng kể giữa khi một Offer kết thúc và khi ngân hàng tạo Offer mới. Đây chính là "inter-offer delay" mà Traditional PM không thể đo được.

Đây là câu hỏi **KHÔNG THỂ** trả lời bằng Traditional PM vì Traditional PM không có khái niệm "từ Offer N sang Offer N+1".

Khi một Offer bị từ chối/hủy, ngân hàng cần trung bình 1.1 ngày để tạo Offer tiếp theo! Đây là bottleneck ẩn mà ta có thể tối ưu hoá bằng cách tự động tạo offer mới khi 1 offer cũ bị từ chối cũng là 1 cách tăng hiệu suất của thời gian.

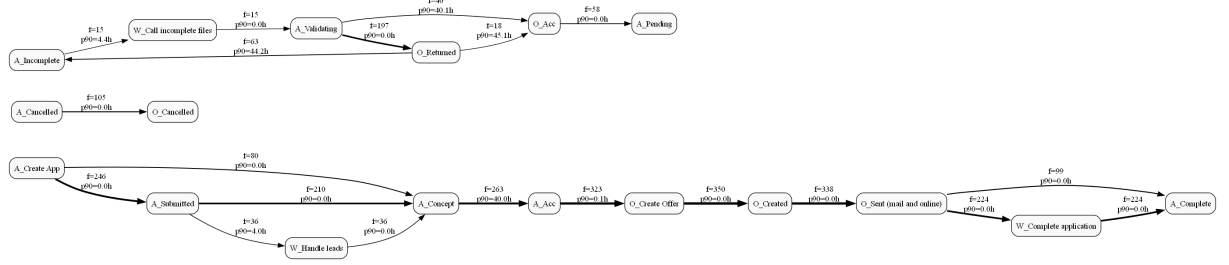
3.4.2 Xây dựng Directly-Follows Graph (DFG) thông qua PM4PY

Từ tập dữ liệu đã tinh gọn, thư viện **PM4PY** được sử dụng để nội suy đồ thị trực tiếp (DFG). Để tối ưu hóa trực quan, một **Thuật toán Cắt tỉa (Pruning Algorithm)** được áp dụng:

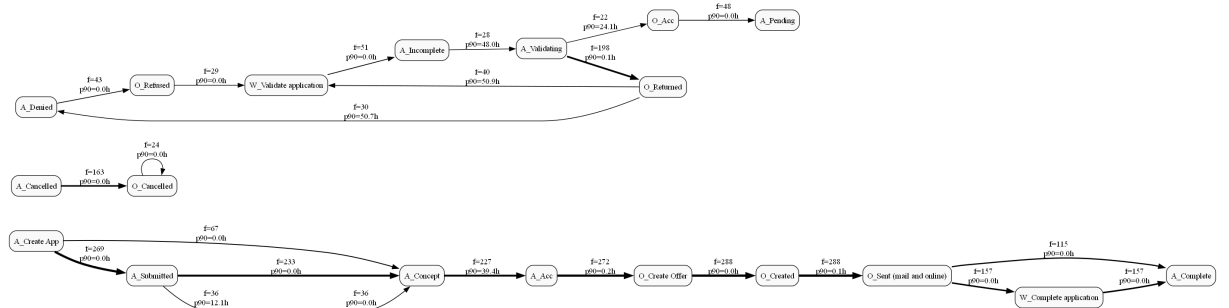
- **Lọc tần suất cạnh (min_edge_freq):** Lập bảng băm (Hash map) lưu trữ các cặp cạnh nối tiếp nhau ($\text{src} \rightarrow \text{tgt}$) và loại bỏ bất kỳ cạnh nào có tần suất xuất hiện dưới mức ngưỡng (ví dụ: < 15 lần).
- **Lọc nút nhánh (top_outgoing_per_node):** Tại mỗi nút trạng thái, thuật toán đánh giá trọng số cạnh dựa trên Tần suất và Thời gian chờ P90 (90th percentile), sau đó chỉ giữ lại tối đa 2 hướng đi chính ($\text{outgoing edges} = 2$).

Nhờ quá trình tinh chỉnh thuật toán này, đồ thị DFG sinh ra phản ánh rõ nét luồng xung quanh các điểm drift cùng với hai thông số then chốt trên từng cạnh: Số lần luồng dữ liệu đi qua (Frequency) và Độ trễ thời gian tối đa (P90 Waiting Time).

Chúng ta dựa vào 2 đặc điểm để có thể đánh giá quy trình thay đổi của 1 điểm drift đó chính là **Limit Raise** và **New Credit** ở các tuần thứ 19, 24, 25, 49. Ta quan sát quy trình thay đổi bằng cách xây dựng đồ thị quy trình trước điểm drift và sau điểm drift. Ví dụ ở New Credit ở tuần thứ 19:



Hình 9: Quy trình của New Credit trước W19



Hình 10: Quy trình của New Credit sau W19

Khi đó ta có thể dễ dàng thấy được khác nhau của quy trình trước và sau của các tuần ở cả 2 trạng thái khác nhau, tránh được lỗi spaghetti.

4 Các bài toán mở rộng

4.1 Bài toán cảnh báo sớm

Bộ dữ liệu được phân chia theo tỷ lệ 8:2 để phục vụ công việc train và test. Hồ sơ được thực hiện dự đoán kết quả sau 8 bước đầu tiên, không quá nhỏ để phòng ngừa sự thiếu thông tin, không quá lớn để không bị data leakage.

4.1.1 Mục tiêu

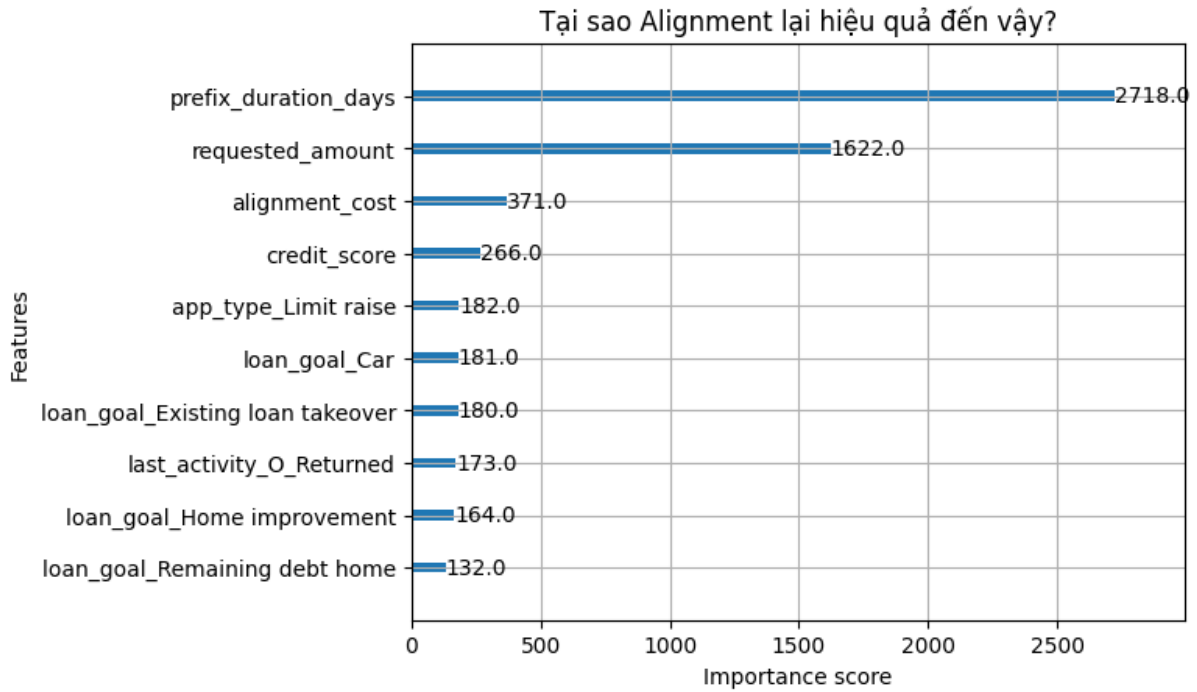
Sau khi khai phá được quy trình, ta có thể nhận thấy việc ảnh hưởng của một quy trình bị tắc nghẽn sẽ gây thiệt hại như thế nào cho bộ máy. Do đó, ta phát triển bài toán một hồ sơ khi đi được một vài bước đầu sẽ có dự đoán kết quả ra sao.

4.1.2 Phương pháp

Trước tiên nhóm xử lý data để tìm ra 10 quy trình đẹp nhất trong bộ dữ liệu. Sau đó nhóm sử dụng phương pháp Alignment để tính được chi phí sửa đổi (sự khác nhau giữa quy trình hiện tại và 10 quy trình chuẩn trên) để làm 1 feature. Cùng với đó nhóm sử dụng các features khác như:

- Đặc trưng động: prefix duration days, last activity
- Đặc trưng tĩnh: requested amount, credit score, loan goal, app type
- Đặc trưng Tuân thủ/Độ lệch: fitness

- Bản chất của Alignment: Hãy tưởng tượng ngân hàng có một quy trình chuẩn (Reference Model) được vẽ trên giấy. Tuy nhiên, trong thực tế (Event Log), nhân viên đôi khi làm sai: họ làm thiếu bước, làm thừa bước, hoặc làm sai thứ tự. Thuật toán Alignment hoạt động giống như một phép so sánh ADN. Nó đặt chuỗi sự kiện thực tế của một hồ sơ cạnh chuỗi quy trình chuẩn, và cố gắng "căn chỉnh" sao cho hai chuỗi này khớp với nhau nhất, với chi phí sửa đổi (Cost) là thấp nhất.



Hình 11: Tại sao lại cần Alignment

Ta có thể thấy trong các features thì alignment cost đóng vai trò quan trọng thứ 3. Chứng tỏ feature này có vai trò đáng kể. Sau đó model được sử dụng để huấn luyện là XGBoost để dự đoán kết quả của hồ sơ

4.2 Thuật toán xây dựng Happy Path

Quy trình tìm Happy Path trải qua 5 bước như:

1. Tiền xử lý và chuẩn hoá sự kiện: chỉ giữ lại các sự kiện có **lifecycle = complete**, đồng nhất lại định dạng timestamp.
2. Bóc tách các nhóm có hồ sơ thành công.
3. Thuật toán chấm điểm và tìm Happy Path: Gom các hồ sơ thành công thành các Variants và chấm điểm chúng dựa trên ma trận 2 chiều: Độ bao phủ và Kết quả (Variants nào đảm bảo được vừa phổ biến, vừa xử lý nhanh gọn) sẽ được chọn làm happy paths.



Hình 12: Ví dụ về 1 happy path

Các Happy Paths này vừa cho chúng ta biết về 1 quy trình hoàn hảo không bị khựng là như nào vừa có thể sử dụng nó cho quá trình PLG, giả lập dữ liệu để chuẩn bị cho định hướng sau này.

5 Kết quả

5.1 Bài toán cảnh báo sớm

Class	Sử dụng alignment			Không sử dụng alignment		
	Precision	Recall	F1-score	Precision	Recall	F1-score
Denied	0.58	0.61	0.60	0.39	0.12	0.18
Success	0.97	0.89	0.93	0.74	0.89	0.81
Churn	0.57	0.77	0.66	0.81	0.73	0.77
Accuracy	0.83			0.74		
Macro avg F1	0.73			0.59		
Weighted avg F1	0.84			0.72		

Bảng 1: So sánh hiệu năng giữa hai mô hình phân loại

Nhận xét:

- Sử dụng alignment cho thấy hiệu năng tổng thể tốt hơn rõ rệt, với accuracy đạt 0.83 so với 0.74 của không sử dụng alignment. Điều này cũng được phản ánh nhất quán ở macro F1-score (0.73 so với 0.59) và weighted F1-score (0.84 so với 0.72), cho thấy nó không chỉ tốt hơn ở lớp lớn mà còn cân bằng hơn giữa các lớp.
- Xét theo từng lớp, sử dụng alignment vượt trội ở lớp Denied, nơi không sử dụng alignment có recall rất thấp (0.12). Điều này cho thấy không sử dụng alignment gần như không nhận diện được lớp Denied, dẫn đến hiệu suất rất yếu ở nhóm này. Đây là điểm yếu nghiêm trọng vì recall thấp thường đồng nghĩa với bỏ sót nhiều trường hợp quan trọng.
- Ở lớp Success, cả hai mô hình đều hoạt động khá tốt, nhưng sử dụng alignment vẫn nhỉnh hơn về precision (0.97 so với 0.74) và F1-score (0.93 so với 0.81), cho thấy khả năng dự đoán chính xác cao hơn.
- Đối với lớp Churn, không sử dụng alignment lại nhỉnh hơn về precision (0.81 so với 0.57), nhưng sử dụng alignment vẫn cân bằng hơn khi xét cả precision-recall-F1, trong đó recall của sử dụng alignment cao hơn (0.77 so với 0.73), giúp bắt được nhiều trường hợp churn hơn.
- Ta thấy ở cả 2 đều có kết quả tốt ở những trường hợp có lượng lớn data là Success nhưng khi sử dụng alignment lại cải thiện vấn đề ít data ở các lớp nhỏ.

6 Kết luận và Hướng phát triển

1. **Hạn chế và hướng phát triển trong tương lai:** Do giới hạn của tập dữ liệu BPI Challenge 2017 (chỉ phản ánh quy trình của một ngân hàng trong một khung thời gian cụ thể thay vì lịch sử dữ liệu dài hạn), dự án hiện tại mới chỉ trích xuất được một số điểm gãy (Change Points) hữu hạn. Do đó, định hướng phát triển trong

tương lai là sử dụng các Đường dẫn chuẩn (Happy Paths) đã khai phá làm gốc để mô phỏng (simulate) thêm dữ liệu nhân tạo. Tập dữ liệu mở rộng này sẽ được dùng để huấn luyện các mô hình dự báo tiên tiến hơn; hướng tới khả năng tiên đoán sớm các mốc thời gian dị thường và nguyên nhân gốc rễ sinh ra Nút thắt (Bottleneck). Nhờ vậy, ngân hàng có thể chủ động có các kịch bản né tránh, tối ưu hóa nguồn lực và ứng phó kịp thời ngay cả trước khi sự cố quá tải thực sự xảy ra. Cùng với đó với việc tìm ra được dấu hiệu trước mỗi khi drift đã mở ra một hướng phát triển dự báo dấu hiệu đó trước cả khi điều đó xảy ra nhưng cần đảm bảo rằng vấn đề dữ liệu khi mà mỗi năm chỉ có một vài điểm bất thường như vậy, ta cần giả lập dữ liệu hoặc thu thập trong thời gian dài tại nhiều địa điểm/doanh nghiệp khác nhau.

2. **Kết luận về dự án:** Nhìn chung, dự án đã giải quyết trọn vẹn các mục tiêu trọng điểm: Giúp người đọc thấu hiểu bức tranh vận hành đa chiều đằng sau tập dữ liệu; bóc tách chính xác nguyên nhân dẫn đến sự quá tải và đình trệ của ngân hàng; trực quan hóa sự biến động của luồng quy trình xoay quanh các điểm bất thường (Concept Drift); và minh chứng được tính hiệu quả của việc ứng dụng Học máy (Machine Learning) vào bài toán cảnh báo sớm rủi ro hệ thống